

bit_analysis_3m

July 10, 2025

1 Análisis de eficiencia de bits 3 metros de distancia

Este notebook analiza qué tan parecidos son los bits recibidos `bits_rx` con los bits transmitidos `bits_tx` y presenta gráficas, métricas de rendimiento y las imágenes que no cumplen con el criterio de coincidencia.

```
[24]: import pandas as pd
from pathlib import Path
from IPython.display import display, Image
import matplotlib.pyplot as plt

path = "./results/3m/"
nombre_csv = "resultados_comparacion_3m.csv"
df = pd.read_csv(path + nombre_csv)

# Vista rápida del DataFrame
display(df.head())
print(f"Filas totales: {len(df)}")
```

	num_auto	bw_plate_tx \
0	1	dataset_yolo/autos_bw/auto001.png
1	1	dataset_yolo/autos_bw/auto001.png
2	1	dataset_yolo/autos_bw/auto001.png
3	1	dataset_yolo/autos_bw/auto001.png
4	1	dataset_yolo/autos_bw/auto001.png

	bw_plate_rx	ocr_txt_tx	bits_equal	bit_errors \
0	data_3m/autos_bw/auto001_rep01.png	K0Z.808	True	0
1	data_3m/autos_bw/auto001_rep02.png	K0Z.808	True	0
2	data_3m/autos_bw/auto001_rep03.png	K0Z.808	True	0
3	data_3m/autos_bw/auto001_rep04.png	K0Z.808	True	0
4	data_3m/autos_bw/auto001_rep05.png	K0Z.808	True	0

	bits_tx	bits_rx
0	[0 0 1 ... 0 0 0]	[0 0 1 ... 0 0 0]
1	[0 0 1 ... 0 0 0]	[0 0 1 ... 0 0 0]
2	[0 0 1 ... 0 0 0]	[0 0 1 ... 0 0 0]
3	[0 0 1 ... 0 0 0]	[0 0 1 ... 0 0 0]

```
4  [0 0 1 ... 0 0 0]  [0 0 1 ... 0 0 0]
```

Filas totales: 1698

1.1 Distribución de errores de bit por imagen

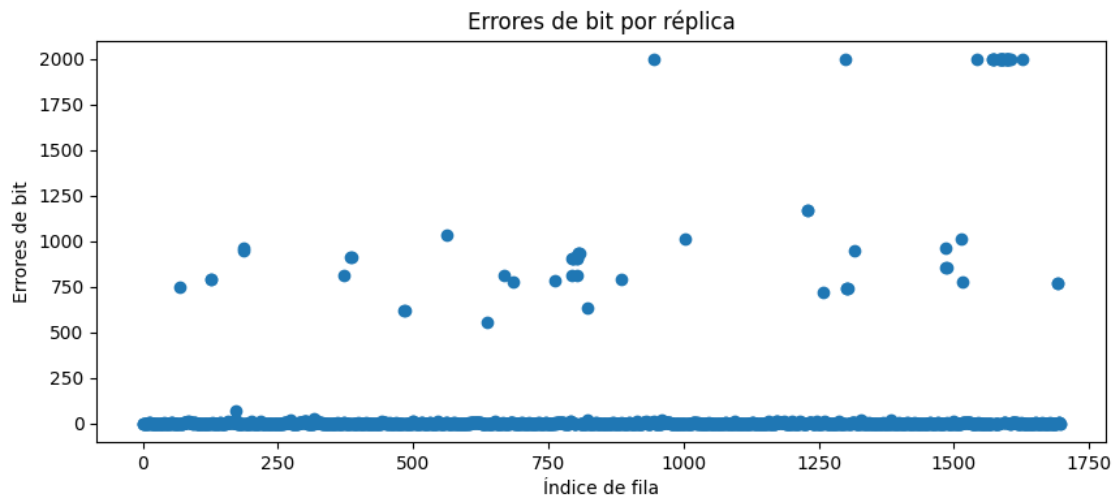
```
[25]: plt.figure()  
df['bit_errors'].plot(kind='hist', bins=30)  
plt.xlabel('Errores de bit por imagen')  
plt.ylabel('Frecuencia')  
plt.title('Histograma de errores de bit')  
plt.show()
```



1.2 Errores de bit por réplica

```
[26]: plt.figure(figsize=(10,4))  
plt.plot(df.index, df['bit_errors'], marker='o', linestyle='none')  
plt.xlabel('Índice de fila')  
plt.ylabel('Errores de bit')  
plt.title('Errores de bit por réplica')
```

```
plt.show()
```



1.3 Imágenes que no cumplen el criterio

```
[27]: # Eliminamos las que tienen 2000 errores, ya que es la imagen inversa
df = df[df['bit_errors'] < 2000]
```

```
[28]: import numpy as np    # por si no estaba importado

# --- Parámetros de tolerancia ---
BITS_TOTAL = 2000
max_errors = 100 # máximo de bits erróneos permitidos
max_ber     = max_errors / BITS_TOTAL # % BER máximo

# --- BER por fila ---
df['bits_total'] = BITS_TOTAL
df['ber'] = df['bit_errors'] / BITS_TOTAL

# --- Criterio de falla ---
fail_df = df[(df['bit_errors'] > max_errors) | (df['ber'] > max_ber)]

print(f'Imágenes que superan el umbral: {len(fail_df)}')
print(f'Total de imágenes: {len(df)}')
print(f'Porcentaje de fallos: {len(fail_df) / len(df) * 100:.2f}%')
display(
    fail_df[['num_auto', 'bw_plate_tx', 'bw_plate_rx',
             'bit_errors', 'ber']]
)
```

Imágenes que superan el umbral: 42

Total de imágenes: 1674
 Porcentaje de fallos: 2.51%

	num_auto	bw_plate_tx \		bw_plate_rx	bit_errors	ber
68	64	dataset_yolo/autos_bw/auto064.png				
126	64	dataset_yolo/autos_bw/auto064.png				
127	64	dataset_yolo/autos_bw/auto064.png				
186	61	dataset_yolo/autos_bw/auto061.png				
187	53	dataset_yolo/autos_bw/auto053.png				
371	105	dataset_yolo/autos_bw/auto105.png				
385	106	dataset_yolo/autos_bw/auto106.png				
386	106	dataset_yolo/autos_bw/auto106.png				
483	61	dataset_yolo/autos_bw/auto061.png				
484	61	dataset_yolo/autos_bw/auto061.png				
561	89	dataset_yolo/autos_bw/auto089.png				
637	77	dataset_yolo/autos_bw/auto077.png				
667	36	dataset_yolo/autos_bw/auto036.png				
684	90	dataset_yolo/autos_bw/auto090.png				
761	89	dataset_yolo/autos_bw/auto089.png				
793	125	dataset_yolo/autos_bw/auto125.png				
794	93	dataset_yolo/autos_bw/auto093.png				
795	93	dataset_yolo/autos_bw/auto093.png				
796	93	dataset_yolo/autos_bw/auto093.png				
802	93	dataset_yolo/autos_bw/auto093.png				
803	125	dataset_yolo/autos_bw/auto125.png				
804	93	dataset_yolo/autos_bw/auto093.png				
805	127	dataset_yolo/autos_bw/auto127.png				
806	127	dataset_yolo/autos_bw/auto127.png				
807	127	dataset_yolo/autos_bw/auto127.png				
822	95	dataset_yolo/autos_bw/auto095.png				
884	125	dataset_yolo/autos_bw/auto125.png				
1002	122	dataset_yolo/autos_bw/auto122.png				
1229	130	dataset_yolo/autos_bw/auto130.png				
1230	130	dataset_yolo/autos_bw/auto130.png				
1259	141	dataset_yolo/autos_bw/auto141.png				
1301	154	dataset_yolo/autos_bw/auto154.png				
1302	154	dataset_yolo/autos_bw/auto154.png				
1303	154	dataset_yolo/autos_bw/auto154.png				
1315	33	dataset_yolo/autos_bw/auto033.png				
1484	32	dataset_yolo/autos_bw/auto032.png				
1485	16	dataset_yolo/autos_bw/auto016.png				
1486	32	dataset_yolo/autos_bw/auto032.png				
1514	96	dataset_yolo/autos_bw/auto096.png				
1515	64	dataset_yolo/autos_bw/auto064.png				
1691	26	dataset_yolo/autos_bw/auto026.png				
1692	26	dataset_yolo/autos_bw/auto026.png				

68	data_3m/autos_bw/auto064_rep01.png	748	0.3740
126	data_3m/autos_bw/auto064_rep02.png	789	0.3945
127	data_3m/autos_bw/auto064_rep03.png	790	0.3950
186	data_3m/autos_bw/auto061_rep01.png	962	0.4810
187	data_3m/autos_bw/auto053_rep01.png	946	0.4730
371	data_3m/autos_bw/auto105_rep01.png	812	0.4060
385	data_3m/autos_bw/auto106_rep01.png	912	0.4560
386	data_3m/autos_bw/auto106_rep02.png	910	0.4550
483	data_3m/autos_bw/auto061_rep02.png	621	0.3105
484	data_3m/autos_bw/auto061_rep03.png	621	0.3105
561	data_3m/autos_bw/auto089_rep01.png	1032	0.5160
637	data_3m/autos_bw/auto077_rep01.png	558	0.2790
667	data_3m/autos_bw/auto036_rep08.png	810	0.4050
684	data_3m/autos_bw/auto090_rep01.png	775	0.3875
761	data_3m/autos_bw/auto089_rep02.png	787	0.3935
793	data_3m/autos_bw/auto125_rep01.png	813	0.4065
794	data_3m/autos_bw/auto093_rep01.png	902	0.4510
795	data_3m/autos_bw/auto093_rep02.png	902	0.4510
796	data_3m/autos_bw/auto093_rep03.png	902	0.4510
802	data_3m/autos_bw/auto093_rep04.png	902	0.4510
803	data_3m/autos_bw/auto125_rep02.png	813	0.4065
804	data_3m/autos_bw/auto093_rep05.png	902	0.4510
805	data_3m/autos_bw/auto127_rep01.png	931	0.4655
806	data_3m/autos_bw/auto127_rep02.png	931	0.4655
807	data_3m/autos_bw/auto127_rep03.png	934	0.4670
822	data_3m/autos_bw/auto095_rep01.png	631	0.3155
884	data_3m/autos_bw/auto125_rep03.png	794	0.3970
1002	data_3m/autos_bw/auto122_rep01.png	1015	0.5075
1229	data_3m/autos_bw/auto130_rep01.png	1173	0.5865
1230	data_3m/autos_bw/auto130_rep02.png	1171	0.5855
1259	data_3m/autos_bw/auto141_rep01.png	719	0.3595
1301	data_3m/autos_bw/auto154_rep01.png	740	0.3700
1302	data_3m/autos_bw/auto154_rep02.png	740	0.3700
1303	data_3m/autos_bw/auto154_rep03.png	740	0.3700
1315	data_3m/autos_bw/auto033_rep10.png	946	0.4730
1484	data_3m/autos_bw/auto032_rep15.png	857	0.4285
1485	data_3m/autos_bw/auto016_rep13.png	959	0.4795
1486	data_3m/autos_bw/auto032_rep16.png	854	0.4270
1514	data_3m/autos_bw/auto096_rep14.png	1013	0.5065
1515	data_3m/autos_bw/auto064_rep10.png	779	0.3895
1691	data_3m/autos_bw/auto026_rep11.png	773	0.3865
1692	data_3m/autos_bw/auto026_rep12.png	773	0.3865

1.3.1 Visualización de las primeras 5 imágenes fallidas

```
[ ]: path = 'results/3m/'

for _, row in fail_df.head(25).iterrows():
    print(f"Auto {row['num_auto']} - errores: {row['bit_errors']}")
    display(Image(filename=row['bw_plate_tx']))
    display(Image(filename=path + row['bw_plate_rx']))
```

Auto 64 - errores: 748



Auto 64 - errores: 789



Auto 64 - errores: 790



Auto 61 - errores: 962



Auto 53 - errores: 946



Auto 105 - errores: 812



Auto 106 - errores: 912



Auto 106 - errores: 910



Auto 61 - errores: 621



Auto 61 - errores: 621





Auto 89 - errores: 1032



Auto 77 - errores: 558



Auto 36 - errores: 810



Auto 90 - errores: 775



Auto 89 - errores: 787



Auto 125 - errores: 813



Auto 93 - errores: 902





Auto 93 - errores: 902



Auto 93 - errores: 902



Auto 93 - errores: 902



Auto 125 - errores: 813



Auto 93 - errores: 902



Auto 127 - errores: 931



Auto 127 - errores: 931



Auto 127 - errores: 934



```
[30]: # --- Umbral de bits erróneos ---
thr_err = 15

# --- Agrupar por auto y comprobar si alguna réplica cumple ---
auto_status = (
    df.groupby('num_auto')['bit_errors']
      .apply(lambda s: (s < thr_err).any())    # True = tiene al menos una buena
)

# Autos sin réplica "buena"
failed_autos = auto_status[~auto_status]      # False significa que falló
num_failed   = failed_autos.size
num_total    = auto_status.size

print(f'Autos sin ninguna réplica con < {thr_err} errores de bit: '
      f'{num_failed} de {num_total}')

# Mostrar la lista (si quieres verla)
display(failed_autos.index.tolist())
```

Autos sin ninguna réplica con < 15 errores de bit: 0 de 182

□